



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

软件体系结构

第13章 体系结构选择与软件开发

赵庆玲

南京理工大学计算机科学与工程学院

体系结构驱动的软件开发简介

(1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

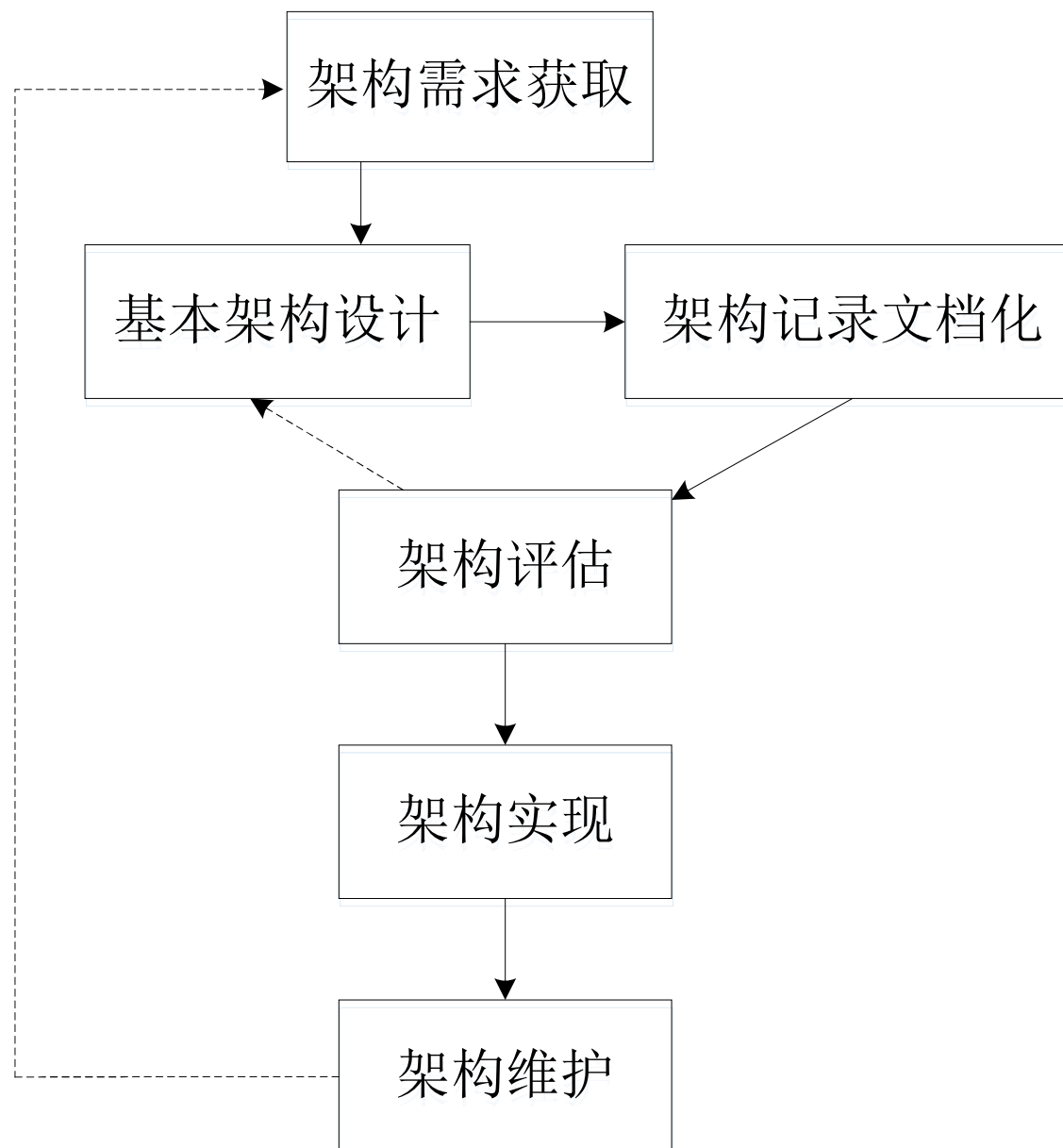
- 体系结构驱动的软件开发主要包括以下几个步骤
 - 1) 体系结构需求获取;
 - 2) 基本体系结构设计;
 - 3) 体系结构记录文档化;
 - 4) 体系结构评估;
 - 5) 体系结构实现;
 - 6) 体系结构维护。

体系结构驱动的软件开发简介

(2)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



体系结构驱动的软件开发流程图



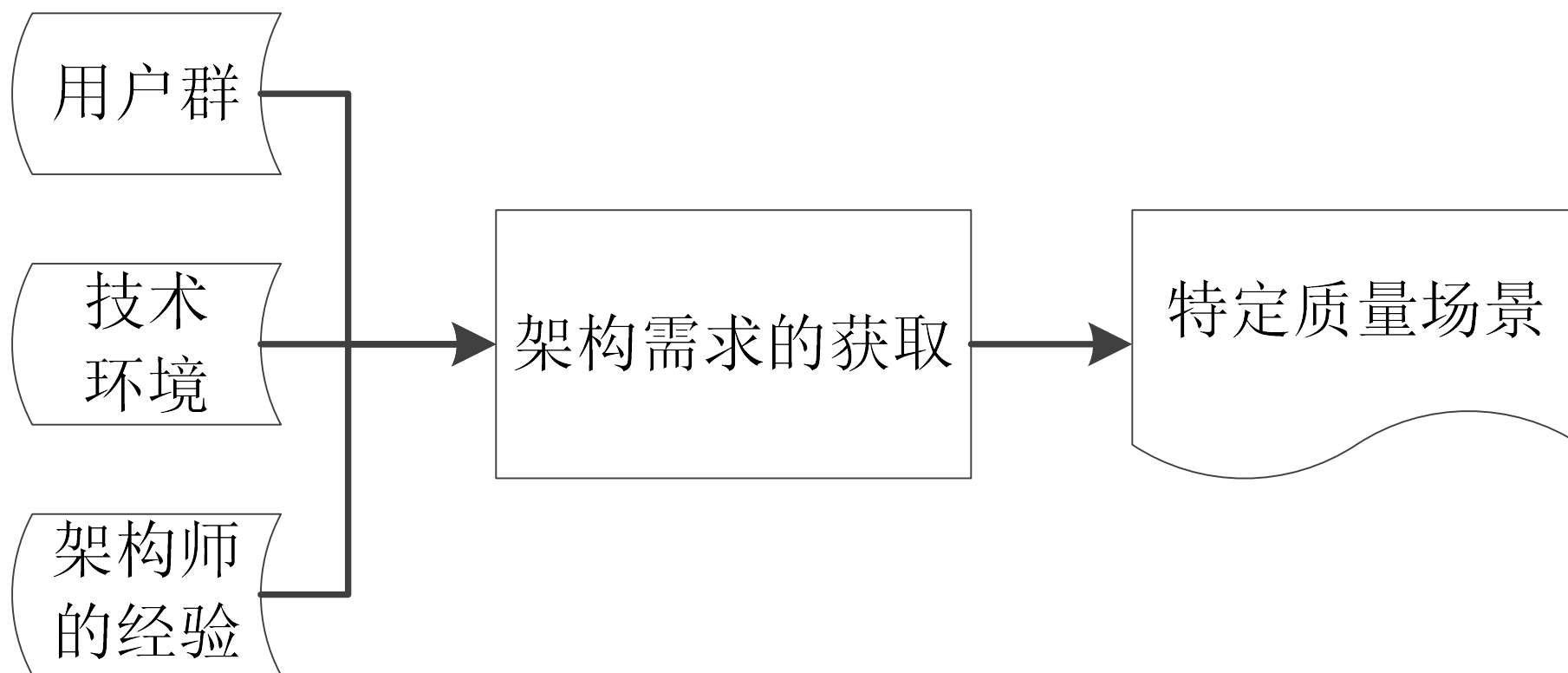
体系结构需求获取（1）

- 体系结构相关需求的获取是从软件的用户群和软件的使用环境中提取的，同时如何抽取恰当的体系结构相关的需求信息，还与参与设计的体系结构师的经验密不可分。
- 体系结构的需求与功能的需求是不同的。在设计体系结构的时候只需要讨论体系结构层次的需求，没有必要再深入到功能性需求。
- 体系结构的需求源于：系统的质量目标、系统的业务目标、软件的利益相关者

体系结构需求获取（2）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



体系结构需求的获取阶段任务描述图

体系结构需求获取（3）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 通常体系结构的需求是十分抽象的，没有什么编程语言会提供实现体系结构需求的机制。
- 因此，进行体系结构需求描述的步骤通常为：
 - 首先要将体系结构的需求通过一定的场景进行描述，接着通过一定的模型描述这些场景，获得体系结构需求的结构化描述。

体系结构需求获取（4）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 1. 质量场景

- 体系结构需求要用质量场景进行描述。
 - 抽象场景：根据软件的使用进行一定层面的分类（如：软件流水线方式、三层结构等），这些分类就会对相应软件提出一定的需求，此类需求即为体系结构需求的抽象场景。
 - 例如：由Yinzer系统体系结构需求的质量场景描述（见下页），可以看出该描述可用于所有用到浏览器和客户端的软件系统的体系结构分析中。

对Yinzer系统体系结构需求的一个完整的质量场景的描述

源	Yinzer用户
触发	请求Yinzer服务器上的网页
环境	正常操作
制品	整个系统
响应	服务器返回网页
响应测试	Yinzer系统在1s内返回页面
完整地质量场景	Yinzer用户点击浏览器中的链接；浏览器向Yinzer系统发送请求，Yinzer系统在1s内返回页面。

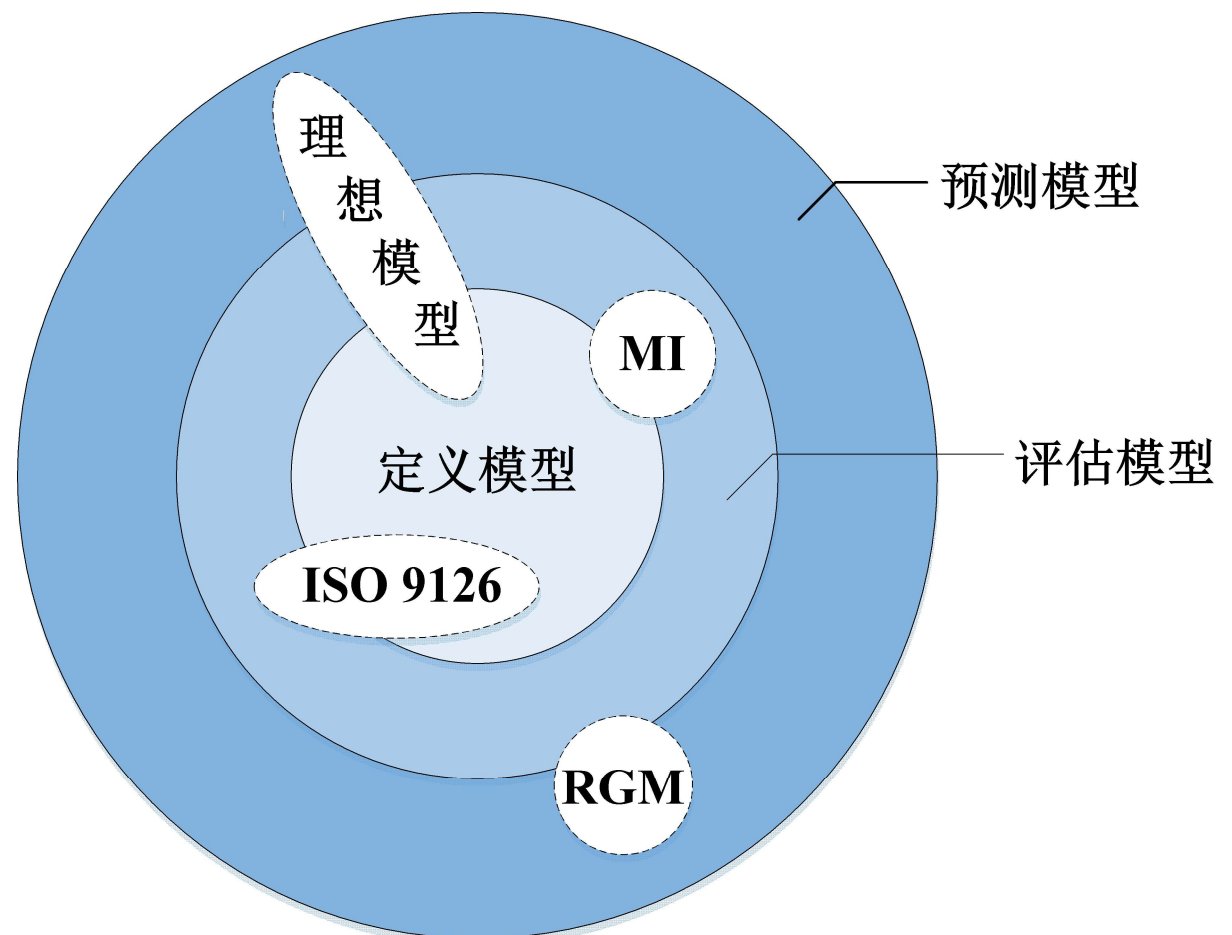
- 1. 质量场景

- 体系结构需求要用质量场景进行描述。

- 对于体系结构师和领域专家来说，需要做的是从抽象场景描述中获得特定的质量属性场景。
 - 例如：“Yinzer用户点击浏览器中的链接；浏览器向Yinzer系统发送请求，**Yinzer系统在1s内返回页面**”就对系统性能提出了要求。
 - 通常来说，我们考虑的特定的质量场景是对性能、可移植性、可替换性、可重用性等质量属性产生影响时的质量场景。

- 2. 软件质量模型

- 软件质量理想模型：可以用来描述，评估和预测质量属性的模型。



软件质量理想模型结构图

- 2. 软件质量模型

- 软件质量元模型（软件质量理想模型设计原则）：
可以清晰地描述质量模型中元素和元素之间相互关系的模型。
- 软件质量理想模型框架：描述了如何将软件质量元模型进行实例化，并且可以对质量属性进行定义、评估、预测，切实提高软件质量属性的框架。

- 3.将质量场景映射到软件质量理想模型
 - 通常，我们获得的质量场景都是以文本的形式呈现的，将质量场景映射到质量模型中需要对质量场景描述的内容进行分解和细化。
 - 对于质量场景中那些很抽象层次的描述我们可以将其规约为抽象场景质量模型，对于特定的质量场景也会将其规约到特定质量的模型中。



软件质量元模型结构图

- 体系结构设计、文档化和评估是一个迭代过程
 - 体系结构设计、文档化和评估是一个迭代过程也是体系结构驱动的软件开发的核心所在。
 - 其步骤如下：
 - 1. 基本体系结构设计
 - 2. 体系结构文档化
 - 3. 体系结构评估

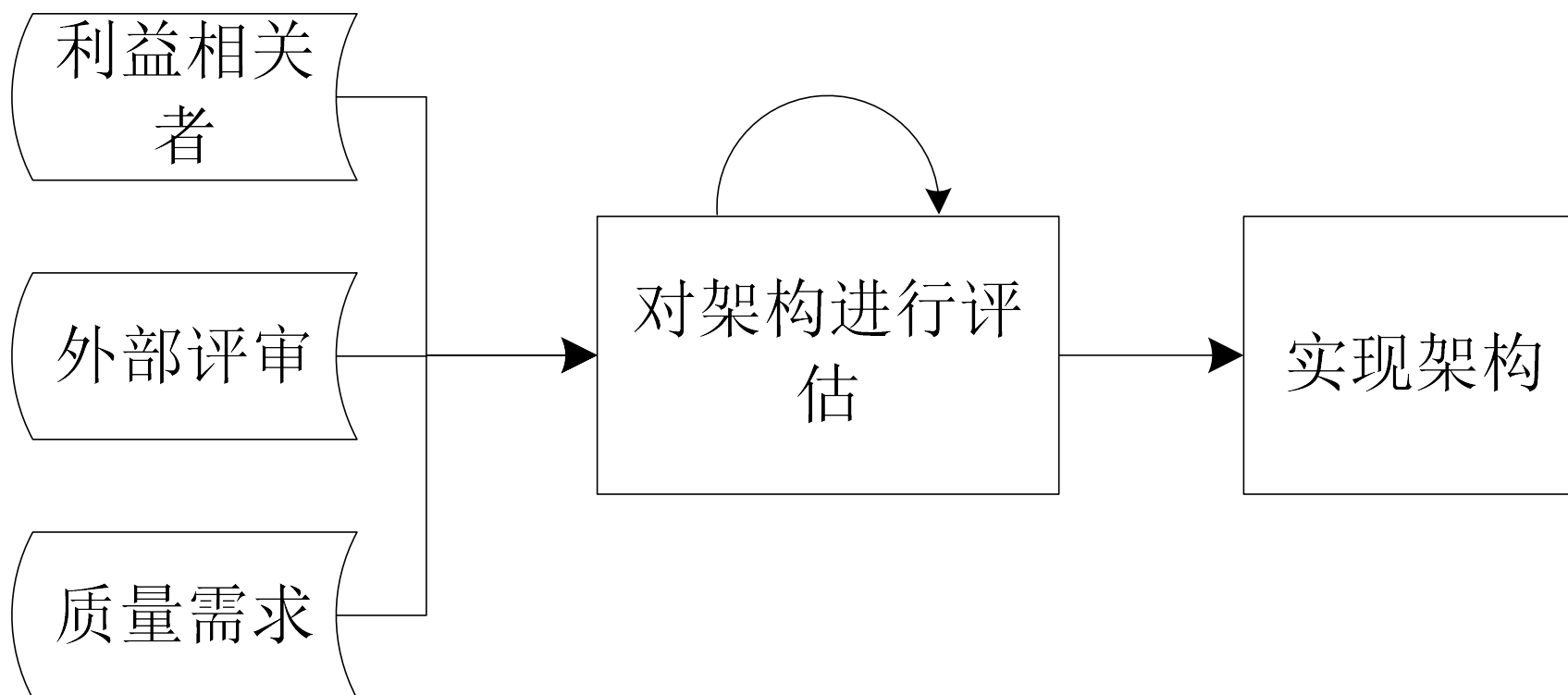
- 1.基本体系结构设计
 - 通过获得的体系结构需求信息，体系结构师对体系结构进行设计并通过文档进行记录.
 - 从零开始的软件系统开发的基本体系结构设计过程：
 - （1）通过功能性需求列表，抽取出体系结构需求列表和类功能列表。
 - （2）选择进行开发的子系统，将其对应的设计子系统记录到基于功能的体系结构中。
 - （3）通过基于功能的体系结构完成并发体系结构的设计。

• 2.体系结构文档化

- 体系结构的设计文档可以加强软件系统的利益相关者之间的联系，从而确定出满足需求的软件体系结构，旨在方便程序员和分析师的工作。
 - 软件体系结构的设计文档必须是完整，并且可以追溯的。
 - 整个体系结构设计文档必须要包含一个显著的起点，同时所有的子系统也必须要链接成一个完整的集合，并提供相应的指针指向子系统内部的详细设计文档。
 - 在体系结构设计文档中应该有一个预先规定的约束框架。使得软件能够在性能，容错性，可维护性，安全性等方面能够有良好的表现。
 - 体系结构设计文档要向所有的利益相关者公开。

• 3.体系结构评估

- 评估的目的是分析体系结构以识别潜在风险并验证设计中已经满足的质量需求。



对体系结构进行评估阶段任务描述图

什么是体系结构的结构 (1)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构的结构
 - 通过一定的结构对软件的体系结构进行描述，把这样的结构称为体系结构的结构。
 - **体系结构的结构**描述了体系结构的基本信息，也包括了类，方法，对象，文件，库等所有需要人做的设计和编码。
 - **体系结构视图**是由体系结构派生而出的，它可以是体系结构的子部分，也可以是多个体系结构信息的综合。

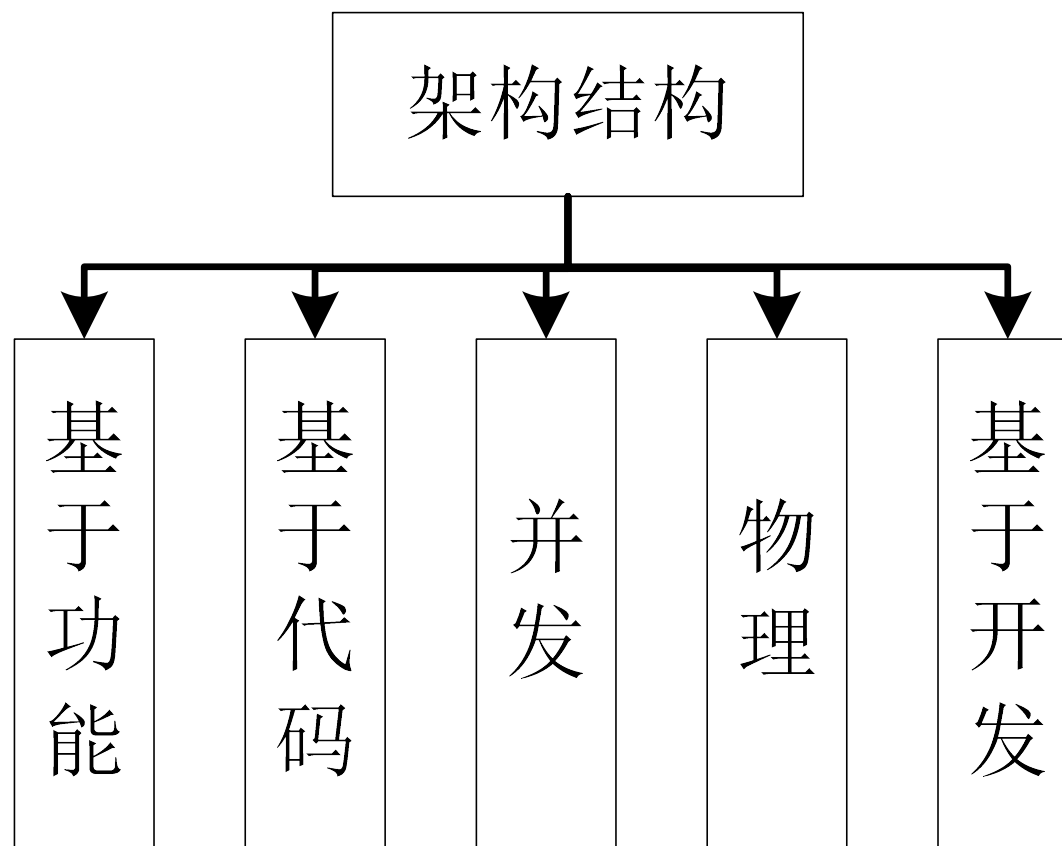
什么是体系结构的结构 (2)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构分类

- 对体系结构的描述有很多种方法，其中有五种最基本的类型



什么是体系结构的结构 (3)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构分类

- 基于功能的结构

- 对软件系统的功能性需求的分解和描述。
 - 在此类体系结构中，“组件”是功能（域）实体；“连接件”是数据的传输实体。
 - 这种结构有助于我们理解功能实体之间的交互，对软件系统的功能性需求有更加深刻的理解，从而设计出满足功能性需求的软件体系结构。

什么是体系结构的结构 (4)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构的分类
 - 基于代码的体系结构
 - 基于代码的体系结构对软件设计过程中关键性代码的抽象描述。
 - 在基于代码的体系结构中，“组件”是包，类，对象，过程，函数，方法等，用于在各种抽象级别封装功能的载体；“连接件”可以使控制流，传输流，共享流，调用关系，对象的实例等。
 - 这种结构对于了解系统的可维护性，可修改性，可重用性和可移植性至关重要。

什么是体系结构的结构 (5)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构的分类

- 并发体系结构

- 并行体系结构与软件系统的并发逻辑性有关，包含着系统的线程和/或进程的数量，执行时间和优先级等信息。
 - 在并行体系结构中，“组件”是可以细化为线程或者进程的并发单元；“连接件”包括有同步，优先级，数据的传输和互斥。
 - 这种结构对软件系统的运作有着至关重要的作用，同时也有助于实现软件系统的安全性和可靠性。

什么是体系结构的结构 (6)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构的分类
 - 物理体系结构
 - 物理体系结构涉及中央处理器（CPU），存储器，总线，网络和输入/输出设备。
 - 可以清晰了解软件系统的可用性，带宽，容量等信息。

什么是体系结构的结构 (7)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构的分类
 - 基于开发的体系结构
 - 开发体系结构涉及文件信息和目录信息。
 - 这种体系结构对于软件系统的管理和控制有着重要的作用，同时也可作为团队分工的重要依据。

什么是体系结构的结构 (8)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

不同的体系结构与软件质量属性之间的影响关系表

质量属性	体系结构
性能	并发体系结构, 物理体系结构
安全性	并发体系结构, 基于代码的体系结构
可靠性/可用性	并发体系结构, 物理体系结构
可修改性/可维护性	基于功能的不同的体系结构与软件质量属性之间的影响关系表体系结构, 基于代码的体系结构, 基于开发的体系结构

从体系结构需求出发的评估（1）

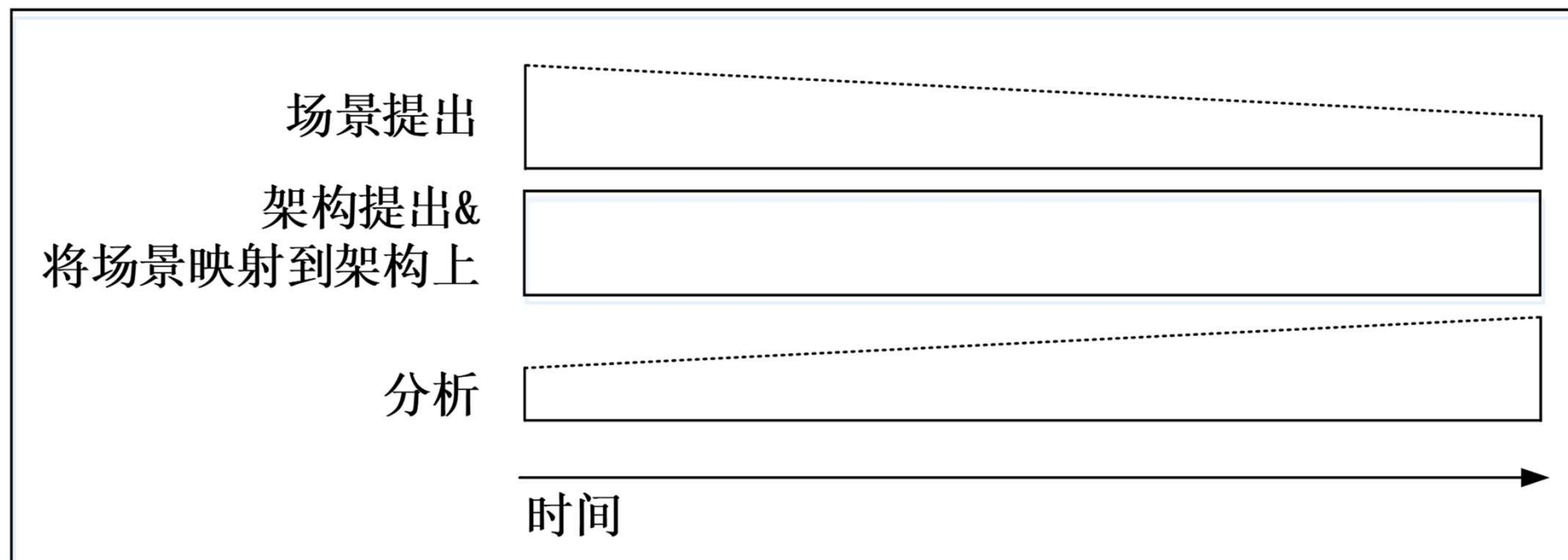


南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构权衡分析法（ATAM: Architecture Tradeoff Analysis Method）
- ATAM的方法来进行权衡评估是目的是：当需要满足某一质量需求时，会使得到其他的质量需求变差，我们需要权衡这种改变是否值得。

- ATAM的过程

- 一个完整地ATAM通常会花费三个整天，其中每天都要进行如下三个过程：场景提出、体系结构提出和将场景映射到体系结构上，并进行分析评估。



ATAM活动及其重要性对比图



寻找ATAM中的“权衡点”和“敏感点”

- 在ATAM中需要评估和确定的问题区域被称为“敏感点”和“权衡点”。
- 敏感点：是体系结构中对实现特定质量场景至关重要的组件的集合。
- 权衡点：是对实现多个质量需求至关重要的敏感点。
- 可通过AHP(层次分析过程) 寻找ATAM中的“权衡点”和“敏感点”

- 对“权衡点”的寻找
 - 寻找“权衡点”的目的是为了在两种或多种质量属性之间进行权衡
 - 可以通过绝对值和相对值两种方式来进行衡量
- 对“敏感点”的寻找
 - 寻找“敏感点”的目的是使设计师搞清楚实现质量目标时应该更关注哪种质量需求。

- 案例

- 某分布式软件系统的体系结构中，假设采用了以下四种可能的体系结构形式为基础：（1）三层结构的J2EE(THTJ)；（2）三层结构的.Net(THTD)；（3）两层的结构(TWOT)；（4）支持分布式代理的平台结构(COAB)。
- 该系统中考虑的质量属性有：可修改性（modifiability），可扩展性（scalability），性能（performance），成本（cost），开发投入（development effort），可移植性（portability）和易安装性（ease of installation）。



寻找ATAM中的“权衡点”和“敏感点” (4)

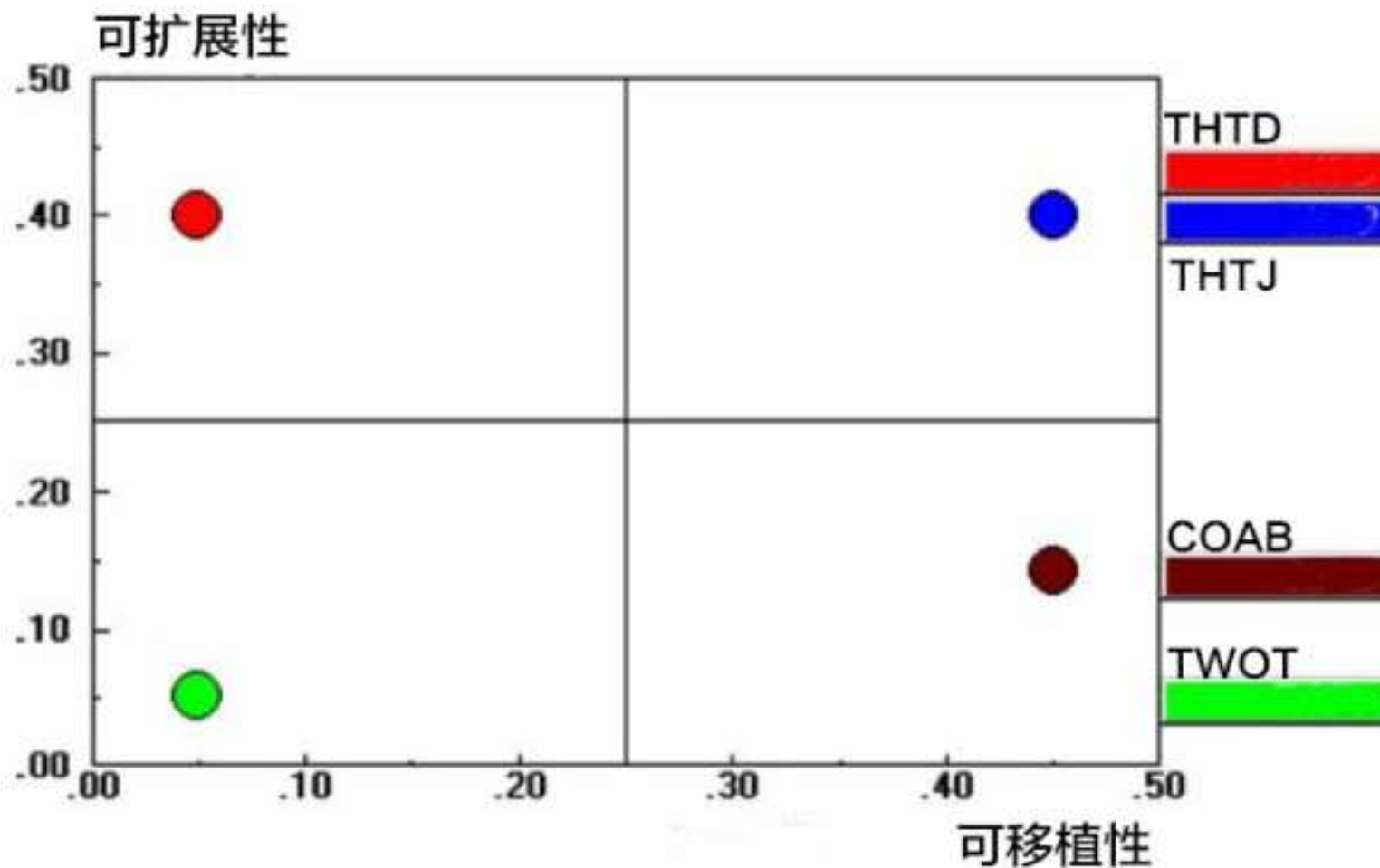
- 案例

- 通过绝对值和相对值来进行权衡点的寻找：
 - 为了实现体系结构所要求的可移植的质量需求，这就导致了数据的传输性能需要从4500TPS（每秒的传输速率）下降到3000TPS，下降的值1500TPS可以作为性能和可移植性这两个质量需求“权衡”值。
 - 更多的情况，我们是无法求出这种绝对值的，因此采用的是相对值来进行表示。为了支持更高的可修改性需求，我们不得不降低性能质量需求，因此确定性能对可修改性的“折衷”值为0.346。

寻找ATAM中的“权衡点”和“敏感点” (5)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

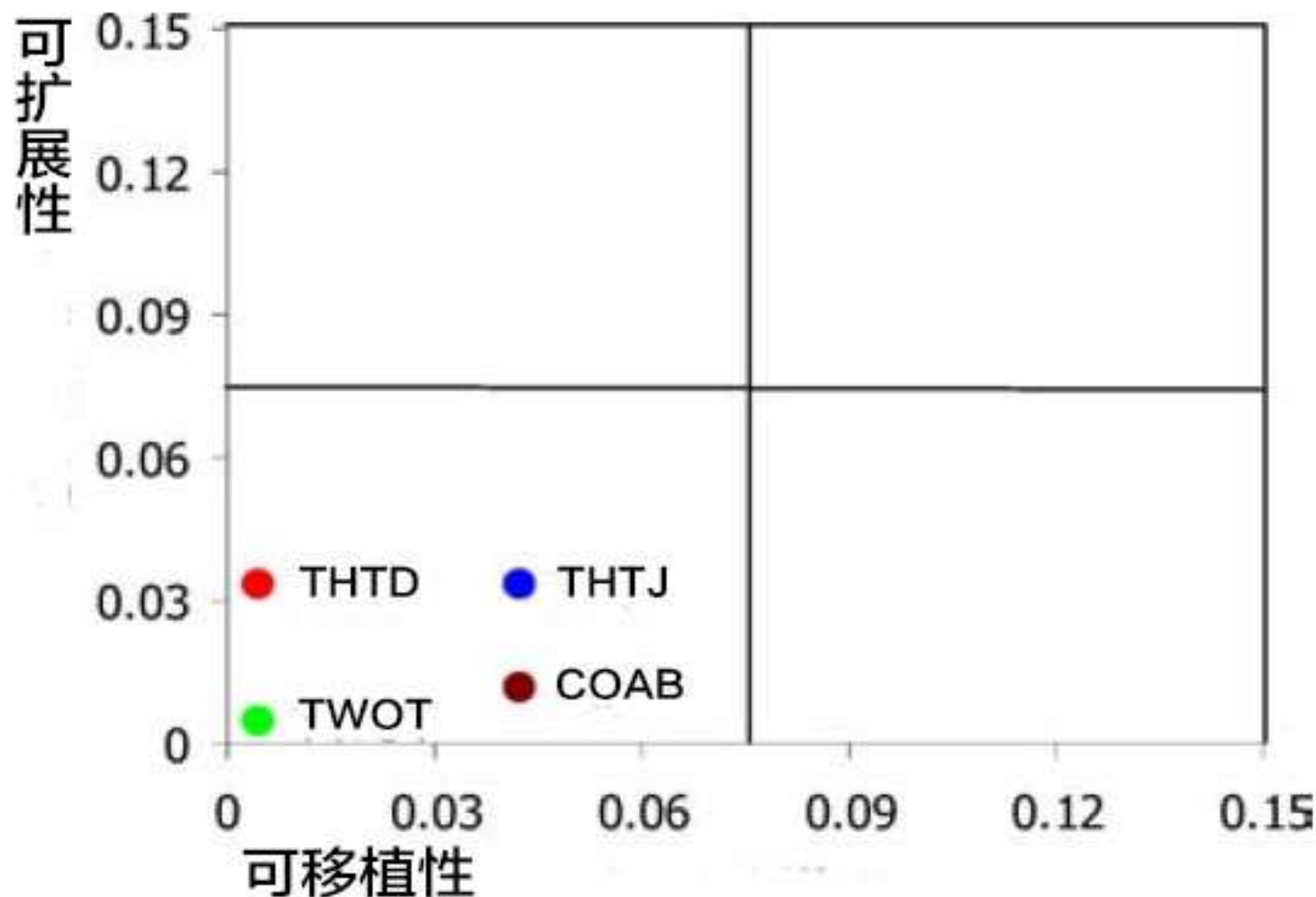


可移植性对可扩展性“权衡点”的结果示意图

寻找ATAM中的“权衡点”和“敏感点” (6)



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



考虑质量需求权值后可移植性对可扩展性的“权衡点”结果示意图

寻找ATAM中的“权衡点”和“敏感点”

对分布式软件系统进行“敏感点”寻找后的结果

质量需求	候选决定	候选决定	最小的影响值
性能	TWOT	COAB	9.4
成本	TWOT	COAB	5.1
开发投入	TWOT	COAB	3.1
可移植性	TWOT	COAB	2.4
易安装性	TWOT	COAB	13.5
可扩展性	COAB	THTD	5.7
可修改性	COAB	TWOT	3.9

目前软件设计方法的主要缺点



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

经验
决定一切



- – 管道-过滤器风格、批处理风格已经非常少见；
– 过程控制风格、黑板结构、虚拟机风格往往针对具体的应用领域；
– OO的思想已经融合在几乎所有的体系结构之中，而事件风格、层次化的思想同样也被广泛使用；
– MVC、基于规则的系统、B/S、数据库风格也是经常使用的风格。

- A.绝大多数实际运行的系统都是几种体系结构的复合：
 - 在系统的某些部分采用一种体系结构而在其他的部分采用另外的体系，故而需要将复合几种基本体系结构组合起来形成复合体系结构。
 - 在实际的系统分析和设计中，首先将整个系统作为一个功能体进行分析和权衡，得到适宜的、最上层的体系结构；
 - 如果该体系结构中的元素较为复杂，可以继续进行分解，得到某一部分的局部体系结构。
- B.将焦点集中在系统的总体结构的考虑上，而避免较多的考虑使用的语言、具体的技术等实现细节上。

风格选择的基本思想： 由粗到细的分解



基本的分析方法：功能和复杂性的分解

- 横向分解：分层次
- 纵向分解：子系统、模块
- 针对每一个分解后得到的模块，根据该问题领域的特性，选择行为模式(具体的体系结构风格)
- 构件、连接件设计
 - 从大粒度风格入手，逐渐细化

- - A.技术因素:
 - (1) Which kinds of components and connectors are used in the style (何种构件、连接件)
 - (2) How control is shared, allocated, and transferred among the components (在运行时, 构件之间的控制机制是如何被共享、分配和转移)
 - (3) How data is communicated through the system (数据如何通讯)
 - (4) How data and control interact (数据与控制如何交互)

- B.性能因素:
 - (5) How the performance will be (性能如何)
- Modifiability (可修改性)
 - Change in Algorithm (算法的变化)
 - Change in Data Representation (数据表示 方式的变化)
 - Enhancement to system function (系统功 能的可扩展性)
- Performance (性能):
 - Both space and time (时空复杂性)
- Reusability (可复用性)

基于经验的SA风格选择原则



A.使用分层结构的经验法则

层次化的思想在任何系统中都可能得到应用；

B.使用数据流风格的经验法则(1)

If your problem can be decomposed into sequential stages, consider batch sequential or pipeline architectures. (如果问题可分解为连续的几个阶段，那么考虑使用顺序批处理风格或管道-过滤器风格)

– If in addition each stage is incremental, so that later stages can begin before earlier stages finish, consider a pipeline architecture. (如果每个阶段是递增的，从而后序可在前序完成之前开始，那么就要使用管道-过滤器风格)

– If your problem involves transformations on continuous streams of data (or on very long streams), consider a pipeline architecture. (如果问题涉及到连续数据流上的转换，考虑使用管道-过滤器风格)

- C.使用OO和仓库风格的经验法则
如果核心问题是应用程序中数据的理解、管理与表示，那么考虑使用仓库或者ADT/OO风格)
 - 如果数据格式的表示可能发生变化，ADT/OO可限制这种变化所影响的范围)
 - 如果数据是持久存在的，使用仓库结构)
- 如果使用仓库风格且输入数据是无序的、执行次序无法预先确定，使用黑板结构)
- 如果使用仓库风格且执行次序由请求来决定、数据是高度结构化的，适用数据库风格
 - 如果追求数据存取的性能，考虑使用分布式数据库风格。

- D.使用交互式风格的经验法则
 - 如果任务之间的控制流可预先设定、无须配置，那么考虑使用主程序-子过程风格、OO风格)
 - (如果任务需要高度的灵活性与可配置性、松散耦合性，或者任务是被动性的，那么考虑使用事件系统或C/S风格)
 - 如果任务的产生者与接收者之间不能预先绑定在一起，使用基于事件的风格)

基于经验的SA风格选择原则



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- – 如果任务分为生产者与消费者，考虑C/S风格)
 - 如果追求客户端的计算效率，考虑胖客户端的C/S风格
 - 如果客户端频繁发生变化，考虑瘦客户端的C/S风格(或B/S风格)
 - 如果服务器端的计算压力过大，考虑使用服务器的集群风格
 - 如果无须中央服务器，使用点对点(P2P)风格

- E.使用虚拟机风格的经验法则

If you have designed a computation but have no machine on which you can execute it, consider an interpreter architecture. (如果设计了某种计算，但没有机器可以支持它运行，那么考虑使用虚拟机/解释器体系结构)

If you have to implement some business logics which are often changed, consider a rule-based architecture. (如果要实现一些经常发生变化的业务逻辑，考虑使用基于规则的系统)

“经验决定一切”带来的问题

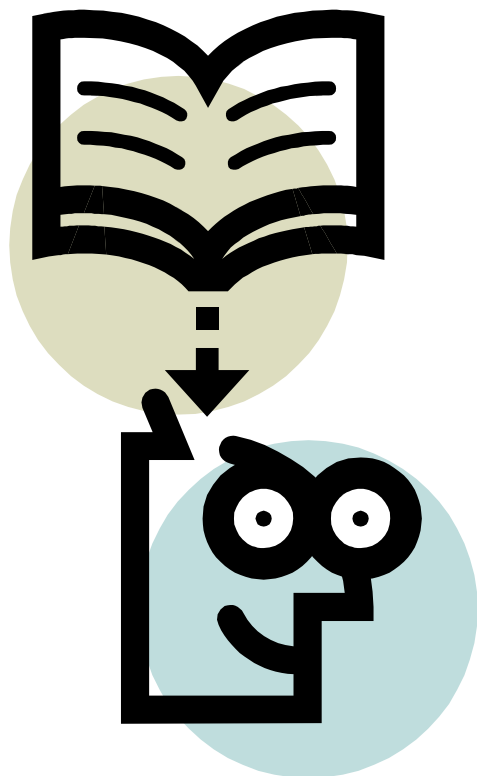


- 经验很重要，没有不行；
- 经验的随意性很大，很难科学地评价各种经验的正确性；
- 没经过高度概括和总结的经验需要后来者花费很长的时间和精力才能继承，甚至可能失传；
- 经验经常限制开拓。

“体系选择矩阵法”的目标



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



- 让选择体系结构的过程变得简单！
- 让选择体系结构的方法变得易学！
- 让每一个程序员都能轻松驾驭体系结构！

“体系选择矩阵法”的基本思想

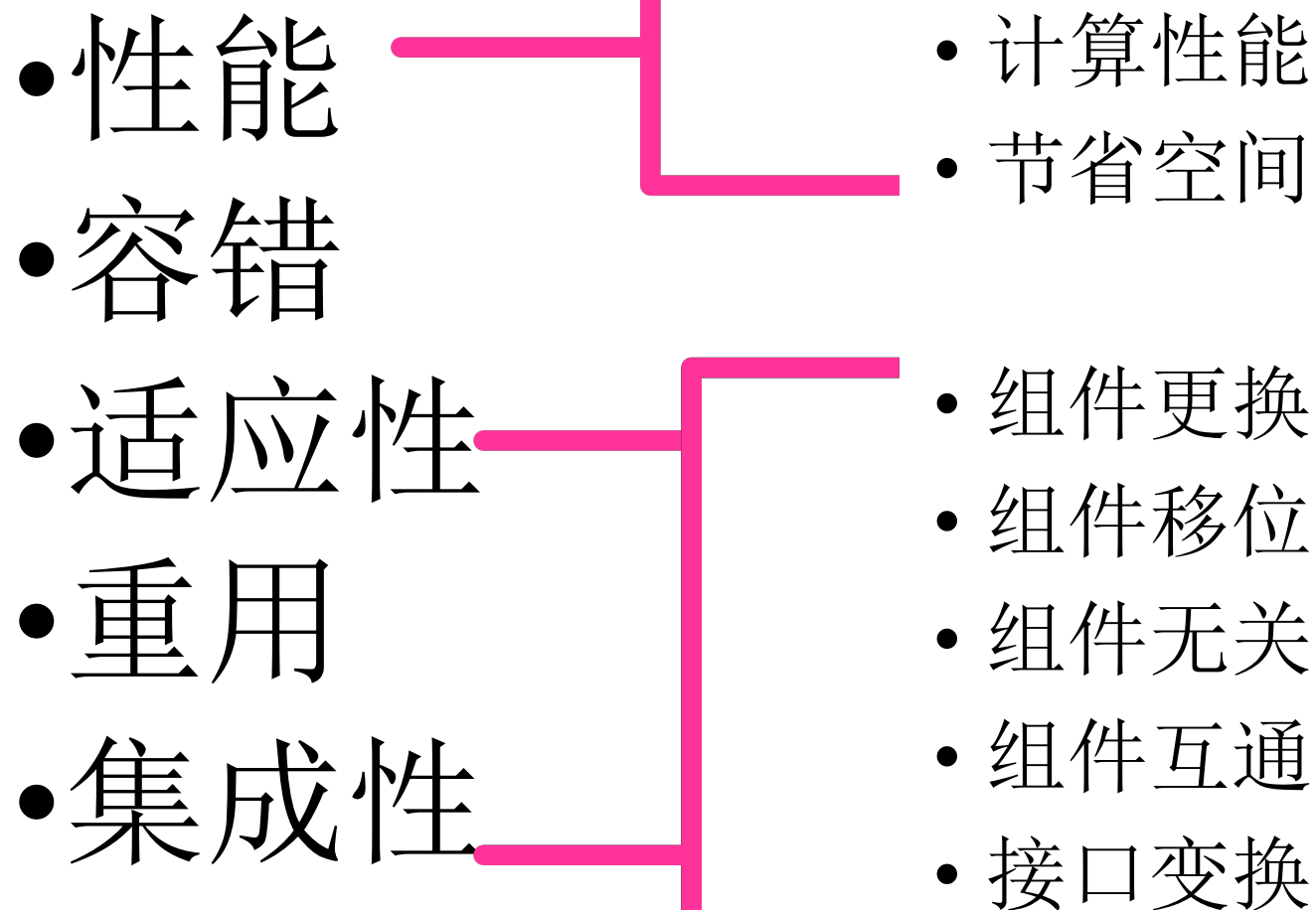


南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

预先评估各种体系结构风格对质量需求的表现，并为它们评分。分数最高的体系结构风格获得录用。



- 结构
 - 因为组件之间的关联，影响的质量
 - 比如：重用、适应性
- 实现
 - 采用不同的实现方法影响的质量
 - 比如：安全、兼容性



各种体系风格对质量需求的影响



	组件重用	组件更换	组件移位	组件无关	组件互通	接口变换	计算性能	节省空间	容错
管道	5	5	1	5	5	2	5	1	1
结构化	2	3	5	3	5	1	5	5	1
面向对象	5	5	5	1	5	1	4	5	1
事件驱动	5	5	5	5	2	1	4	4	5
层次	3	5	1	4	5	4	2	1	1
仓库	5	5	5	3	4	1	4	5	2
客户—服务器	5	5	5	3	5	1	3	5	1
对等	5	5	5	1	5	1	4	3	5

编译器对体系结构质量的期望值



- 期望值的取值范围是0-4，表示对某项体系需求的关心程度。

组件重用: 4

接口变换: 4

组件更换: 4

计算性能: 1

组件移位: 0

节省空间: 0

组件无关: 0

容 错: 0

组件互通: 2

用期望值看体系结构风格的表现



	组件重用	组件更换	组件移位	组件无关	组件互通	接口变换	计算性能	节省空间	容错
期望值	4	4	0	0	2	4	1	0	0
管道	5	5	1	5	5	2	5	1	1
结构化	20	20	0	0	10	8	5	0	0
面向对象	3	5	1	1	2	4	3	3	1
事件驱动	5	5	5	3	2	1	5	5	2
层次	5	5	5	3	2	1	3	5	1
仓库	5	5	5	1	5	1	5	3	5
客户—服务器	5	5	5	1	5	1	5	3	5
对等	5	5	5	1	5	1	5	3	5

Diagram illustrating the expected values for various architectural styles across different criteria. The table shows scores for styles like 结构化, 面向对象, 事件驱动, 层次, 仓库, 客户—服务器, and 对等. A red bracket groups the scores for 结构化, 面向对象, 事件驱动, 层次, 仓库, 客户—服务器, and 对等, with a red arrow pointing to the value 63, indicating a total score or average.

用期望值看体系结构风格的表现



	组件重用	组件更换	组件移位	组件无关	组件交互	接口变换	计算性能	节省空间	容错
期望值	4	4	0	0	2	4	1	0	0
管道	5	5	1	5	5	2	5	1	1
结构化	2	3	5	3	5	1	5	5	1
面向对象	5	5	5	1	5	1	4	5	1
事件驱动	5	5	5	5	2	1	4	4	5
层次	3	5	1	4	5	4	2	1	1
仓库	5	5	5	3	4	1	4	5	2
客户—服务器	5	5	5	3	5	1	3	5	1
对等	5	5	5	1	5	1	4	3	5

$$C = A \times B \times T$$

体系选择矩阵法



	组件重用	组件更换	组件移位	组件无关	组件互连	接口变换	计算性能	节省空间	容错
期望值	4	4	0	0	2	4	1	0	0
管道	5	5	1	5	5	2	5	1	1
结构化	2	3	5	3	5	1	5	5	1
面向对象	5	5	5	1	5	1	4	5	1
事件驱动	5	5	5	5	2	1	4	4	5
层次	3	5	1	4	5	4	2	1	1
仓库	5	5	5	3	4	1	4	5	2
客户—服务器	5	5	5	3	5	1	3	5	1
对等	5	5	5	1	5	1	4	3	5

$$C = A \times B^T$$

对编译器采用体系选择矩阵法



$$\begin{pmatrix} 5 & 5 & 1 & 5 & 5 & 2 & 5 & 1 & 1 \\ 2 & 3 & 5 & 3 & 5 & 1 & 5 & 5 & 1 \\ 5 & 5 & 5 & 1 & 5 & 1 & 4 & 5 & 1 \\ 5 & 5 & 5 & 5 & 2 & 1 & 4 & 4 & 5 \\ 3 & 5 & 1 & 4 & 5 & 4 & 2 & 1 & 1 \\ 5 & 5 & 5 & 3 & 4 & 1 & 4 & 5 & 2 \\ 5 & 5 & 5 & 3 & 5 & 1 & 3 & 5 & 1 \\ 5 & 5 & 5 & 1 & 5 & 1 & 4 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 4 \\ 4 \\ 0 \\ 0 \\ 2 \\ 4 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 63 \\ 39 \\ 58 \\ 52 \\ 60 \\ 56 \\ 57 \\ 58 \end{pmatrix}$$

63	管道风格
39	结构化风格
58	面向对象风格
52	事件驱动风格
60	层次风格
56	仓库风格
57	客户—服务器
58	对等风格



对编译器采用体系选择矩阵法

	组件重用	组件更换	组件移位	组件无关	组件互通	接口变换	计算性能	节省空间	容错
期望值	4	4	0	0	2	4	1	0	0
管道	5	5	1	5	5	2	5	1	1
结构化	2	3	5	3	5	1	5	5	1
面向对象	5	5	5	1	5	1	4	5	1
事件驱动	5	5	5	5	2	1	4	4	5
层次	3	5	1	4	5	4	2	1	1
仓库	5	5	5	3	4	1	4	5	2
客户—服务器	5	5	5	3	5	1	3	5	1
对等	5	5	5	1	5	1	4	3	5

对编译器采用体系选择矩阵法



- 最后决定的体系结构的体系分为

	组件重用	组件更换	组件移位	组件无关	组件互通	接口变换	计算性能	节省空间	容错
期望值	4	4	0	0	2	4	1	0	0
管道	5	5	1	5	5	2	5	1	1
层次	3	5	1	4	5	4	2	1	1
最后选择	5	5	1	4	3	4	1	1	1

- 选择分：63
- 在接口变换方面多加小心

完美分



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

$$(5 \ 5 \ 5 \ 5 \ 5 \ 5 \ 5 \ 5 \ 5) \times \begin{pmatrix} 4 \\ 4 \\ 0 \\ 0 \\ 2 \\ 4 \\ 1 \\ 0 \\ 0 \end{pmatrix} = (75)$$

当前需求

完美地支持各种质量需求的最理想的体系结构

完美分

- 等级=选择分/完美分*100
- 编译器等级=63/75*100=84

“体系选择矩阵法”的优点



- 支持体系结构风格的混合；
- 使选择体系结构的过程变得科学；
- 便于学习，易于掌握；
- 评价各种体系结构风格，提示开发中将遇到的问题；
- 对寻找新的体系结构有引导作用；
- 适合用CASE工具实现。

实例：可穿戴计算机



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 组件重用：4
- 组件更换：4
- 组件移位：4
- 组件无关：0
- 组件互通：4
- 接口变换：4
- 计算性能：4
- 节省空间：2
- 容错：4

实例：可穿戴计算机



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

$$\begin{pmatrix} 5 & 5 & 1 & 5 & 5 & 2 & 5 & 1 & 1 \\ 2 & 3 & 5 & 3 & 5 & 1 & 5 & 5 & 1 \\ 5 & 5 & 5 & 1 & 5 & 1 & 4 & 5 & 1 \\ 5 & 5 & 5 & 5 & 2 & 1 & 4 & 4 & 5 \\ 3 & 5 & 1 & 4 & 5 & 4 & 2 & 1 & 1 \\ 5 & 5 & 5 & 3 & 4 & 1 & 4 & 5 & 2 \\ 5 & 5 & 5 & 3 & 5 & 1 & 3 & 5 & 1 \\ 5 & 5 & 5 & 1 & 5 & 1 & 4 & 3 & 5 \end{pmatrix} \times \begin{pmatrix} 4 \\ 4 \\ 4 \\ 0 \\ 4 \\ 4 \\ 4 \\ 4 \\ 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 98 \\ 98 \\ 114 \\ 116 \\ 86 \\ 114 \\ 110 \\ 126 \end{pmatrix} \begin{matrix} \text{管道风格} \\ \text{结构化风格} \\ \text{面向对象风格} \\ \text{事件驱动风格} \\ \text{层次风格} \\ \text{仓库风格} \\ \text{客户—服务器} \\ \text{对等风格} \end{matrix}$$

实例：可穿戴计算机



- 完美分=150
- 等级= $126/150*100=84$
- 需要当心接口变换问题
- 可以继续用此方法选择每台计算机上软件的体系结构

“体系选择矩阵法”的缺点



- 一定程度上限制了创新；
- 很多问题还有待进一步细化；
- 没有理论证明；
- 缺少大量的实践考验。

体系结构的实现与维护



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 体系结构驱动的软件开发的特点之一是软件体系结构与开发团队组织结构具有一致性。
- 开发团队的组织结构必须反映到软件体系结构上，反之亦然。